Guia do Projeto Final – Fundamentos de Engenharia de Dados

A cada grupo será atribuído um domínio de atuação: Saúde, E-commerce, Logística, Finanças, Educação, Agro, Mobilidade Urbana. O projeto deve aplicar o mesmo arcabouço técnico independentemente da origem dos dados. O storytelling e as perguntas de negócio serão definidos por cada grupo, respeitando os requisitos funcionais comuns descritos abaixo.

A base de dados selecionada NÃO pode ser nenhuma utilizada previamente pelos membros do grupo nos laboratórios anteriores.

Storytelling (Problema de Negócio):
O grupo deve construir uma história, cenário hipotético. E deve definir algumas perguntas para uma área de negócio que precisa de um painel confiável e atualizado diariamente. Perguntas como:

- Quais produtos geram mais receita e também mais devoluções?
- Qual o tempo médio de entrega por região?
- Existem pedidos com valores inconsistentes (frete maior que o total do pedido)?

Desafio:

O desafio do grupo é construir um pipeline ELT completo, avaliando a qualidade dos dados desde a ingestão (Raw), passando por transformações confiáveis (Silver/Gold), até a visualização. Tudo orquestrado e com rastreabilidade de linhagem.

Fluxo:

1. ~~**Airbyte** → carrega os dados do dataset escolhido para o schema raw do PostgreSQL~~. (OPCIONAL)
2. ~~2.~~ **Script Python Customizado:** Desenvolver um script Python para realizar a extração e carga (EL) dos dados brutos para o PostgreSQL.
3. **Great Expectations** → valida os dados recém-chegados na raw (qualidade na entrada).
4. **dbt** → transforma raw → silver (limpeza, tipagem) → gold (modelo estrela).
5. **Airflow** → orquestra as etapas com tratamento de falhas.
6. **Ferramenta de visualização** → conecta-se ao schema gold para dashboards.
7. **Docker** → para reprodutibilidade.

Lista de Requisitos Técnicos por Ferramenta

Ferramenta	Requisitos Obrigatórios
Airbyte/Script Python	~~- Conector de origem: File (CSV) ou Banco de Dados.~~ ~~- Conector de destino: PostgreSQL (schema raw).~~ ~~- Sincronização full refresh ou incremental.~~ - Script containerizado e orquestrado via Airflow
PostgreSQL	- Arquitetura medalhão: três schemas (raw, silver, gold). - Tabelas raw.* idênticas às fontes. - Tabelas gold apropriadas para as visualizações.
Great Expectations	- Suite de expectativas aplicada exclusivamente sobre o schema raw. - Mínimo 3 expectativas (ex.: expect_column_values_to_not_be_null, expect_column_values_to_be_between). - Checkpoint salvo e executável via CLI ou Python.
dbt	- Modelos stg_* (silver) e modelos dim_* / fact_* (gold). - Pelo menos uma tabela fato e duas dimensões (ex.: fact_orders, dim_customers, dim_products). - Uso de surrogate keys (ex.: dbt_utils.generate_surrogate_key). - Macro customizada (ex.: calcular faixa de valor de frete). - Testes genéricos (unique, not_null, accepted_values) em colunas críticas. - Pelo menos 2 testes singulares. - Documentação (dbt docs generate) com descrições e linhagem.
Apache Airflow	- DAG com dependência explícita: AirbyteSensor → GreatExpectationsOperator → BashOperator (dbt run) → BashOperator (dbt test). - Tratamento de falhas (retry, alerta simples).
Visualização	- Conexão com schema gold. - Pelo menos 2 dashboards
Docker	- Arquivo docker-compose.yml com serviços: postgres, airbyte, airflow, visualização, great-expectations (ou integrado ao Airflow). - Volumes persistentes para dados.

Entregáveis

- Repositório GitHub contendo:
    - docker-compose.yml
    - Diretório airflow/dags/ com a DAG
    - Diretório dbt/ (projeto dbt com modelos, macros, tests, docs)
    - Diretório great_expectations/ (suites, checkpoints)
    - README.md com:
    - Storytelling e perguntas de negócio
    - Diagrama de arquitetura
    - Passo a passo para executar o pipeline
    - Capturas de tela dos dashboards
    - Link para o dataset escolhido e instruções de download (se não estiver versionado)

- Vídeo de demonstração (máx. 7 min) mostrando:
    - Execução do Airflow (UI)
    - Relatório do Great Expectations
    - Linhagem do dbt (lineage graph)
    - Dashboards funcionando

- Documento detalhado de configuração de todo o ambiente, com todos os scripts (comandos) que precisam ser executados para instalação e configuração das ferramentas. Precisa ser executável por uma pessoa que tem uma máquina somente com SO e softwares básicos instalado.
- Um check list da Lista de Requisitos Técnicos por Ferramenta, indicando se cada um dos requisitos foi cumprido integralmente, parcialmente, ou não realizado.
- PPT de apresentação.
- Documento que detalha a contribuição de cada membro do grupo no projeto.

Critérios de Avaliação (Projeto “A”)

Critério	Peso	Descrição do Nível Máximo (Nota A)
Pipeline Completo	25%	Todas as etapas (ingestão → qualidade → transformação → orquestração → visualização) funcionam de ponta a ponta sem intervenção manual. O Airflow dispara automaticamente o pipeline diário.
Modelagem e dbt	20%	Modelo estrela com surrogate keys, macro reutilizável, documentação rica (linhagem, descrições) e testes passando em 100% dos dados de exemplo.
Qualidade e Governança	5%	Great Expectations valida a Raw com suíte robusta e gera relatório de qualidade.
Orquestração	15%	DAG bem estruturada, retry lógico e logs claros. Capacidade de reiniciar a partir de um ponto de falha.
Reprodutibilidade (Docker)	10%	Qualquer colega consegue levantar todo o ambiente com docker-compose up e executar o pipeline em menos de 10 minutos.
Power Point	5%	Apresentação PPT que detalha o passo a passo do processo realizado pelo grupo.
Apresentação/Vídeo demonstração	10%	Apresentação técnica: explicar base de dados escolhida, limitações, problemas de qualidade. Explicar arquitetura implementada e resultados, com gráficos dos dados brutos, relatório de qualidade, linhagem e dashboards. Descrever desafios encontrados e como lidaram com eles.
Documentação	10%	Documento detalhado de configuração de todo o ambiente, com todos os scripts (comandos) que precisam ser executados para instalação e configuração das ferramentas. Precisa ser executável por uma pessoa que tem uma máquina somente com SO e softwares básicos instalado.

Checklist de Auto-Avaliação para o Grupo

- O storytelling descreve claramente o domínio e as perguntas de negócio.
- O Airbyte/Script python carrega os dados para raw sem erros.
- O Great Expectations valida a raw e gera relatório (HTML ou JSON).
- O dbt produz staging, fatos e dimensões com surrogate keys.
- A macro customizada é usada em pelo menos um modelo gold.
- Os testes do dbt (genéricos) passam nos dados de exemplo.
- O Airflow executa toda a DAG com sucesso ou trata falhas explicitamente.
- O dashboard responde às perguntas de negócio.
- O ambiente sobe com docker-compose up e o pipeline roda sem ajustes manuais.

🎥 Extra 1. Vídeo Tutorial Completo (Instalação, Configuração e Execução)

+ 10% na disciplina

Cada grupo poderá produzir um vídeo tutorial que demonstre, na prática, como reproduzir o ambiente do zero. O vídeo deve ser hospedado em plataforma acessível (YouTube não listado ou Google Drive) e o link incluído no README.md.

Conteúdo obrigatório do vídeo:

Segmento	O que mostrar
1. Pré-requisitos	Instalação de Docker, Docker Compose, Git, Python (se necessário) – em sistema limpo (Ubuntu ou WSL2).
2. Clonagem e estrutura	git clone do repositório, explicação da árvore de diretórios (dbt, airflow, great_expectations, docker-compose).
3. Configuração de variáveis de ambiente	Credenciais (ex.: senhas do PostgreSQL, ~~Airbyte~~).
4. Subida do ambiente com Docker	Comando docker-compose up -d e verificação dos containers rodando (docker ps).
~~5. Configuração do Airbyte (UI)~~	~~Instalação e configuração do ambiente. Criação da fonte (dataset escolhido) e destino (PostgreSQL raw), execução manual da 6ync.~~
6. Execução do Great Expectations	Rodar o checkpoint via CLI ou Airflow, mostrar relatório HTML gerado.
7. Execução do dbt	dbt run, dbt test, dbt docs generate e abrir a documentação com lineage graph.
8. Execução da DAG no Airflow	Disparar a DAG, acompanhar logs, ver sucesso das tarefas.
9. Visualização (Metabase/Superset)	Conexão com schema gold, dashboards prontos respondendo às perguntas de negócio.

Critérios de avaliação do vídeo:

- Clareza de áudio e imagem, não precisa de edição profissional, mas deve ser inteligível.
- Execução bem-sucedida do zero – o expectador deve conseguir seguir o vídeo e replicar.
- Narrativa explicativa, não apenas “digite isso”, mas “por que estamos fazendo isso”.
- Inclusão de um erro comum e sua solução, ex.: porta já ocupada, permissão de volume, Airbyte sem conector.

💡 **Dica**: O vídeo pode ser gravado com OBS Studio (gratuito) e editado minimamente para cortar longas esperas (build de imagens, download de dependências).

🏗️ Extra 2 – Infraestrutura como Código com Terraform

+ 10% na disciplina

Os grupos que optarem por este extra deverão provisionar todo o ambiente (ou parte crítica dele) utilizando Terraform, em vez de apenas docker-compose. O objetivo é demonstrar capacidade de definir infraestrutura declarativa, mesmo que em ambiente local, containers Docker gerenciados pelo provedor docker do Terraform.

Requisitos obrigatórios para o extra:

- Arquivos .tf organizados (provedores, variáveis, recursos, outputs).
- Uso de variáveis (ex.: postgres_password, airflow_user) com terraform.tfvars.example.
- Outputs claros (URLs dos serviços, comandos para acessar logs).
- O pipeline de dados (~~Airbyte~~ → Great Expectations → dbt) não precisa ser disparado pelo Terraform – o Terraform cuida apenas da infraestrutura estática. A orquestração continua sendo do Airflow.